Steps & Commands

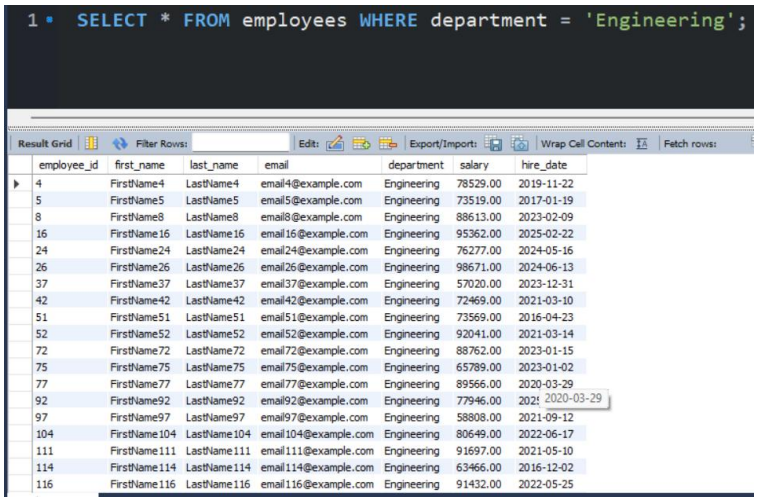
```
CREATE DATABASE performance_lab;

USE performance_lab;


CREATE TABLE employees (

employee_id INT PRIMARY KEY AUTO_INCREMENT,

first_name VARCHAR(50),

last_name VARCHAR(50),

email VARCHAR(100),

department VARCHAR(50),

salary DECIMAL(10, 2),

hire_date DATE

);


DELIMITER //


CREATE PROCEDURE generate_employee_data(IN num_rows INT)

BEGIN

  DECLARE i INT DEFAULT 1;

  DECLARE first_name VARCHAR(50);

  DECLARE last_name VARCHAR(50);

  DECLARE email VARCHAR(100);

  DECLARE department VARCHAR(50);

  DECLARE salary DECIMAL(10, 2);

  DECLARE hire_date DATE;


  WHILE i <= num_rows DO

    SET first_name = CONCAT('FirstName', i);

    SET last_name = CONCAT('LastName', i);

    SET email = CONCAT('email', i, '@example.com');

    SET department = ELT(FLOOR(1 + RAND() * 5), 'Sales', 'Marketing', 'Engineering', 'HR', 'Finance');
```

```
    SET salary = FLOOR(50000 + RAND() * 50000);
    SET hire_date = DATE_SUB(CURDATE(), INTERVAL FLOOR(RAND() * 3650) DAY);

    INSERT INTO employees (first_name, last_name, email, department, salary, hire_date)
    VALUES (first_name, last_name, email, department, salary, hire_date);

    SET i = i + 1;
  END WHILE;
END //

DELIMITER ;

CALL generate_employee_data(10000);
```

Step-by-Step Instructions

Step 1: Execute a Query Without an Index

Command: SELECT * FROM employees WHERE department = 'Engineering';



```
1 • SELECT * FROM employees WHERE department = 'Engineering';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

employee_id	first_name	last_name	email	department	salary	hire_date
4	FirstName4	LastName4	email4@example.com	Engineering	78529.00	2019-11-22
5	FirstName5	LastName5	email5@example.com	Engineering	73519.00	2017-01-19
8	FirstName8	LastName8	email8@example.com	Engineering	88613.00	2023-02-09
16	FirstName16	LastName16	email16@example.com	Engineering	95362.00	2025-02-22
24	FirstName24	LastName24	email24@example.com	Engineering	76277.00	2024-05-16
26	FirstName26	LastName26	email26@example.com	Engineering	98671.00	2024-06-13
37	FirstName37	LastName37	email37@example.com	Engineering	57020.00	2023-12-31
42	FirstName42	LastName42	email42@example.com	Engineering	72469.00	2021-03-10
51	FirstName51	LastName51	email51@example.com	Engineering	73569.00	2016-04-23
52	FirstName52	LastName52	email52@example.com	Engineering	92041.00	2021-03-14
72	FirstName72	LastName72	email72@example.com	Engineering	88762.00	2023-01-15
75	FirstName75	LastName75	email75@example.com	Engineering	65789.00	2023-01-02
77	FirstName77	LastName77	email77@example.com	Engineering	89566.00	2020-03-29
92	FirstName92	LastName92	email92@example.com	Engineering	77946.00	202: 2020-03-29
97	FirstName97	LastName97	email97@example.com	Engineering	58808.00	2021-09-12
104	FirstName104	LastName104	email104@example.com	Engineering	80649.00	2022-06-17
111	FirstName111	LastName111	email111@example.com	Engineering	91697.00	2021-05-10
114	FirstName114	LastName114	email114@example.com	Engineering	63466.00	2016-12-02
116	FirstName116	LastName116	email116@example.com	Engineering	91432.00	2022-05-25

Step 2: Analyze the Query Execution Plan

Command: EXPLAIN SELECT * FROM employees WHERE department = 'Engineering';

```
EXPLAIN SELECT * FROM employees WHERE department = 'Engineering';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	NULL	ALL	NULL	NULL	NULL	NULL	9873	10.00	Using where

Step 3: Create an Index on the department Column

Command: CREATE INDEX idx_department ON employees (department);

```
CREATE INDEX idx_department ON employees (department);
```

Output | Action Output

#	Time	Action	Message	Duration / Fetch
1	19:48:00	CREATE DATABASE performance_lab	1 row(s) affected	0.016 sec
2	19:48:06	USE performance_lab	0 row(s) affected	0.015 sec
3	19:48:17	CREATE TABLE employees (employee_id INT PRIMARY KEY AUTO_INCREMENT, first_name VARCHAR(50...	0 row(s) affected	0.016 sec
4	19:50:56	CREATE PROCEDURE generate_employee_data(IN num_rows INT) BEGIN DECLARE i INT DEFAULT 1	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL ser...	0.000 sec
5	19:52:56	CREATE PROCEDURE generate_employee_data(IN num_rows INT) BEGIN DECLARE i INT DEFAULT 1; ...	0 row(s) affected	0.016 sec
6	19:53:13	CALL generate_employee_data(10000)	1 row(s) affected	9.781 sec
7	19:53:55	SELECT * FROM employees WHERE department = 'Engineering' LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 se
8	19:54:40	EXPLAIN SELECT * FROM employees WHERE department = 'Engineering'	1 row(s) returned	0.000 sec / 0.000 se
9	20:10:18	CREATE INDEX idx_department ON employees (department)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.046 sec

Step 4: Re-analyze the Query Execution Plan

Command: EXPLAIN SELECT * FROM employees WHERE department = 'Engineering';

```
EXPLAIN SELECT * FROM employees WHERE department = 'Engineering';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	NULL	ref	idx_department	idx_department	203	const	2013	100.00	NULL

Step 5: Compare Query Execution Times

Command: SELECT BENCHMARK(1000, (SELECT count(*) FROM employees WHERE department = 'Engineering'));

```
SELECT BENCHMARK(1000, (SELECT COUNT(*) FROM employees WHERE department = 'Engineering'));
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BENCHMARK(1000, (SELECT COUNT(*) FROM employees WHERE department = 'Engineering'))
0

Result 18

Output | Action Output

#	Time	Action	Message	Duration / Fetch
1	20:19:37	SELECT BENCHMARK(1000, (SELECT COUNT(*) FROM employees WHERE department = 'Engineering')) LIM...	1 row(s) returned	0.000 sec / 0.000 sec

Command: SELECT BENCHMARK(1000, (SELECT count(*) FROM employees WHERE department = 'Engineering'));

```
SELECT BENCHMARK(1000, (SELECT COUNT(*) FROM employees WHERE department = 'Engineering'));
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BENCHMARK(1000, (SELECT COUNT(*) FROM employees WHERE department = 'Engineering'))
0

Result 18

Output | Action Output

#	Time	Action	Message	Duration / Fetch
1	20:19:37	SELECT BENCHMARK(1000, (SELECT COUNT(*) FROM employees WHERE department = 'Engineering')) LIM...	1 row(s) returned	0.000 sec / 0.000 sec

Step 6: Experiment with Different Queries

Code (Example Query 1 - No Index):

Command: SELECT * FROM employees WHERE salary > 80000;

```
1    SELECT * FROM employees WHERE salary > 80000;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

employee_id	first_name	last_name	email	department	salary	hire_date
8	FirstName8	LastName8	email8@example.com	Engineering	88613.00	2023-02-09
11	FirstName11	LastName11	email11@example.com	HR	89587.00	2026-04-03
14	FirstName14	LastName14	email14@example.com	Sales	99510.00	2019-09-22
16	FirstName16	LastName16	email16@example.com	Engineering	95362.00	2025-02-22
17	FirstName17	LastName17	email17@example.com	Finance	92535.00	2018-11-07
19	FirstName19	LastName19	email19@example.com	Finance	94526.00	2021-01-21
26	FirstName26	LastName26	email26@example.com	Engineering	98671.00	2024-06-13
31	FirstName31	LastName31	email31@example.com	HR	82403.00	2022-05-31
32	FirstName32	LastName32	email32@example.com	Finance	87754.00	2017-12-24
33	FirstName33	LastName33	email33@example.com	Finance	95443.00	2017-03-14
36	FirstName36	LastName36	email36@example.com	Sales	83985.00	2025-03-15
38	FirstName38	LastName38	email38@example.com	HR	94056.00	2023-07-30
39	FirstName39	LastName39	email39@example.com	HR	84944.00	2022-05-10
41	FirstName41	LastName41	email41@example.com	HR	91702.00	2018-02-08

Command: EXPLAIN SELECT * FROM employees WHERE salary > 80000;

```
1    EXPLAIN SELECT * FROM employees WHERE salary > 80000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	NULL	ALL	NULL	NULL	NULL	NULL	9873	33.33	Using where

Code (Create Index on Salary):

Command: CREATE INDEX idx_salary ON employees (salary);

```
1    CREATE INDEX idx_salary ON employees (salary);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:19:37	SELECT BENCHMARK(1000, (SELECT COUNT(*) FROM employees WHERE department = 'Engineering')) LIM...	1 row(s) returned	0.000 sec / 0.000 sec
2	20:20:22	SELECT BENCHMARK(1000, (SELECT * FROM employees WHERE department = 'Engineering')) LIMIT 0, 1000	Error Code: 1241. Operand should contain 1 column(s)	0.000 sec
3	20:20:31	SELECT BENCHMARK(1000, (SELECT count(*) FROM employees WHERE department = 'Engineering')) LIMIT...	1 row(s) returned	0.000 sec / 0.000 sec
4	20:21:07	SELECT * FROM employees WHERE salary > 80000 LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
5	20:21:38	EXPLAIN SELECT * FROM employees WHERE salary > 80000	1 row(s) returned	0.000 sec / 0.000 sec
6	20:22:11	CREATE INDEX idx_salary ON employees (salary)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec

Code (Re-run Query with Index):

Command: SELECT * FROM employees WHERE salary > 80000;

```
1    SELECT * FROM employees WHERE salary > 80000;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

employee_id	first_name	last_name	email	department	salary	hire_date
4035	FirstName4035	LastName4035	email4035@example.com	Engineering	80006.00	2023-05-13
8932	FirstName8932	LastName8932	email8932@example.com	Sales	80011.00	2019-08-20
8719	FirstName8719	LastName8719	email8719@example.com	Finance	80013.00	2025-11-30
9222	FirstName9222	LastName9222	email9222@example.com	HR	80013.00	2025-11-30
7716	FirstName7716	LastName7716	email7716@example.com	Marketing	80015.00	2016-09-24
4345	FirstName4345	LastName4345	email4345@example.com	Marketing	80018.00	2017-03-11
4558	FirstName4558	LastName4558	email4558@example.com	HR	80023.00	2017-03-17
7483	FirstName7483	LastName7483	email7483@example.com	HR	80026.00	2018-04-17
7672	FirstName7672	LastName7672	email7672@example.com	Finance	80039.00	2024-11-26
5220	FirstName5220	LastName5220	email5220@example.com	Sales	80051.00	2017-11-05
1403	FirstName1403	LastName1403	email1403@example.com	Marketing	80052.00	2016-07-17
9294	FirstName9294	LastName9294	email9294@example.com	HR	80053.00	2018-10-02
9889	FirstName9889	LastName9889	email9889@example.com	Engineering	80056.00	2022-07-30
8038	FirstName8038	LastName8038	email8038@example.com	Engineering	80065.00	2018-08-26
8835	FirstName8835	LastName8835	email8835@example.com	Finance	80069.00	2025-03-10
1775	FirstName1775	LastName1775	email1775@example.com	HR	80073.00	2026-02-14
2311	FirstName2311	LastName2311	email2311@example.com	Finance	80090.00	2020-10-17
9710	FirstName9710	LastName9710	email9710@example.com	HR	80093.00	2018-05-05
3634	FirstName3634	LastName3634	email3634@example.com	Sales	80110.00	2018-01-10

Command: EXPLAIN SELECT * FROM employees WHERE salary > 80000;

```
1   EXPLAIN SELECT * FROM employees WHERE salary > 80000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	NULL	ALL	idx_salary	NULL	NULL	NULL	9873	40.93	Using where

Code (Example Query 2 - No Index):

Command: SELECT * FROM employees WHERE hire_date > '2020-01-01';

```
1   SELECT * FROM employees WHERE hire_date > '2020-01-01';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

employee_id	first_name	last_name	email	department	salary	hire_date
1	FirstName1	LastName1	email1@example.com	Marketing	63917.00	2020-08-01
3	FirstName3	LastName3	email3@example.com	Sales	50326.00	2021-06-02
6	FirstName6	LastName6	email6@example.com	Sales	61304.00	2020-11-30
8	FirstName8	LastName8	email8@example.com	Engineering	88613.00	2023-02-09
9	FirstName9	LastName9	email9@example.com	Marketing	68059.00	2026-01-20
10	FirstName10	LastName10	email10@example.com	Sales	52215.00	2024-09-10
11	FirstName11	LastName11	email11@example.com	HR	89587.00	2026-04-03
13	FirstName13	LastName13	email13@example.com	HR	56797.00	2021-04-03
16	FirstName16	LastName16	email16@example.com	Engineering	95362.00	2025-02-22
18	FirstName18	LastName18	email18@example.com	Sales	78037.00	2022-12-12
19	FirstName19	LastName19	email19@example.com	Finance	94526.00	2021-01-21
22	FirstName22	LastName22	email22@example.com	HR	52685.00	2024-05-21
23	FirstName23	LastName23	email23@example.com	HR	66234.00	2023-05-11
24	FirstName24	LastName24	email24@example.com	Engineering	76277.00	2024-05-16
25	FirstName25	LastName25	email25@example.com	Marketing	63852.00	2023-06-25
26	FirstName26	LastName26	email26@example.com	Engineering	98671.00	2024-06-13
27	FirstName27	LastName27	email27@example.com	Finance	69706.00	2026-03-11
29	FirstName29	LastName29	email29@example.com	HR	61369.00	2024-06-29
31	FirstName31	LastName31	email31@example.com	HR	82403.00	2022-05-31

Command: EXPLAIN SELECT * FROM employees WHERE hire_date > '2020-01-01';

```
1   EXPLAIN SELECT * FROM employees WHERE hire_date > '2020-01-01';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	NULL	ALL	NULL	NULL	NULL	NULL	9873	33.33	Using where

Code (Create Index on Hire Date):

Command: CREATE INDEX idx_hire_date ON employees (hire_date);

```
1   CREATE INDEX idx_hire_date ON employees (hire_date);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:19:37	SELECT BENCHMARK(1000, (SELECT COUNT(*) FROM employees WHERE department = 'Engineering')) LIM...	1 row(s) returned	0.000 sec / 0.000 se
2	20:20:22	SELECT BENCHMARK(1000, (SELECT * FROM employees WHERE department = 'Engineering')) LIMIT 0, 1000	Error Code: 1241. Operand should contain 1 column(s)	0.000 sec
3	20:20:31	SELECT BENCHMARK(1000, (SELECT count(*) FROM employees WHERE department = 'Engineering')) LIMIT...	1 row(s) returned	0.000 sec / 0.000 se
4	20:21:07	SELECT * FROM employees WHERE salary > 80000 LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 se
5	20:21:38	EXPLAIN SELECT * FROM employees WHERE salary > 80000	1 row(s) returned	0.000 sec / 0.000 se
6	20:22:11	CREATE INDEX idx_salary ON employees (salary)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
7	20:23:00	SELECT * FROM employees WHERE salary > 80000 LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 se
8	20:24:35	EXPLAIN SELECT * FROM employees WHERE salary > 80000	1 row(s) returned	0.000 sec / 0.000 se
9	20:26:22	SELECT * FROM employees WHERE hire_date > '2020-01-01' LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 se
10	20:27:05	EXPLAIN SELECT * FROM employees WHERE hire_date > '2020-01-01'	1 row(s) returned	0.000 sec / 0.000 se
11	20:27:31	CREATE INDEX idx_hire_date ON employees (hire_date)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec

Code (Re-run Query with Index):

Command: SELECT * FROM employees WHERE hire_date > '2020-01-01';

```
1    SELECT * FROM employees WHERE hire_date > '2020-01-01';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

employee_id	first_name	last_name	email	department	salary	hire_date
4211	FirstName4211	LastName4211	email4211@example.com	HR	95138.00	2020-01-02
3670	FirstName3670	LastName3670	email3670@example.com	Sales	57184.00	2020-01-03
8208	FirstName8208	LastName8208	email8208@example.com	Engineering	99418.00	2020-01-03
8462	FirstName8462	LastName8462	email8462@example.com	HR	87035.00	2020-01-03
277	FirstName277	LastName277	email277@example.com	Sales	87319.00	2020-01-04
1590	FirstName1590	LastName1590	email1590@example.com	Marketing	67762.00	2020-01-04
4127	FirstName4127	LastName4127	email4127@example.com	Sales	69020.00	2020-01-04
8422	FirstName8422	LastName8422	email8422@example.com	Finance	91658.00	2020-01-04
2678	FirstName2678	LastName2678	email2678@example.com	HR	74448.00	2020-01-05
5069	FirstName5069	LastName5069	email5069@example.com	Sales	86621.00	2020-01-05
9485	FirstName9485	LastName9485	email9485@example.com	Finance	81480.00	2020-01-05
7462	FirstName7462	LastName7462	email7462@example.com	Marketing	97789.00	2020-01-06
4893	FirstName4893	LastName4893	email4893@example.com	Sales	80445.00	2020-01-08
8685	FirstName8685	LastName8685	email8685@example.com	HR	78489.00	2020-01-08
482	FirstName482	LastName482	email482@example.com	HR	75847.00	2020-01-09
9911	FirstName9911	LastName9911	email9911@example.com	HR	99270.00	2020-01-09
4234	FirstName4234	LastName4234	email4234@example.com	Marketing	56257.00	2020-01-10

Command: EXPLAIN SELECT * FROM employees WHERE hire_date > '2020-01-01';

```
1    EXPLAIN SELECT * FROM employees WHERE hire_date > '2020-01-01';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	NULL	ALL	idx_hire_date	NULL	NULL	NULL	9873	63.44	Using where

Summarizing Observations

In this lab, query performance was tested before and after using indexes in a MySQL database. The results showed that queries without indexes used a full table scan (type: ALL), which requires examining all data in the table. This resulted in longer execution times, especially when the data volume was large (10,000 rows). After creating an index on a specific column, such as department, the query used an index scan. The number of rows examined was significantly fewer, resulting in faster query execution times. This was also evident in the results of the BENCHMARK() function, where queries with indexes had significantly lower execution times than those without indexes. Additional tests on the salary and hire_date columns showed a similar pattern. Queries without indexes tended to be slow, but after indexes were created, performance improved significantly.

Conclusions

Indexes play a crucial role in improving database query performance. Using indexes allows the database to access data more quickly without having to perform a full table scan. However, using indexes also has trade-offs: they require additional storage space and can slow down insert, update, and delete processes. Therefore, indexes must be used appropriately to meet the query's needs.